

# OBSERVACIONES DE LA PRÁCTICA

Estudiante 1 Cod 202611361 - Juan Pablo Martínez Rico

Estudiante 2 Cod XXXX

Estudiante 3 Cod XXXX

| Máquina 1         |                                                    |
|-------------------|----------------------------------------------------|
| Procesador        | Intel(R) Core(TM) i5-1035G1 CPU @ 1.00GHz 1.19 GHz |
| Memoria RAM (GB)  | 8.00 GB                                            |
| Sistema Operativo | Windows 11 Home - 64 bits                          |

Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.

## Resultados

### Carga de Catálogo LINEAR PROBING

| Factor de Carga (PROBING) | Consumo de Datos [kB] | Tiempo de Ejecución Real @LP [ms] |
|---------------------------|-----------------------|-----------------------------------|
| 0.1                       | 587664.31             | 17188.36                          |
| 0.5                       | 403232.23             | 10671.39                          |
| 0.7                       | 389901.78             | 8944.91                           |
| 0.9                       | 381471.13             | 23530.31                          |

Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING

### Carga de Catálogo SEPARATE CHAINING

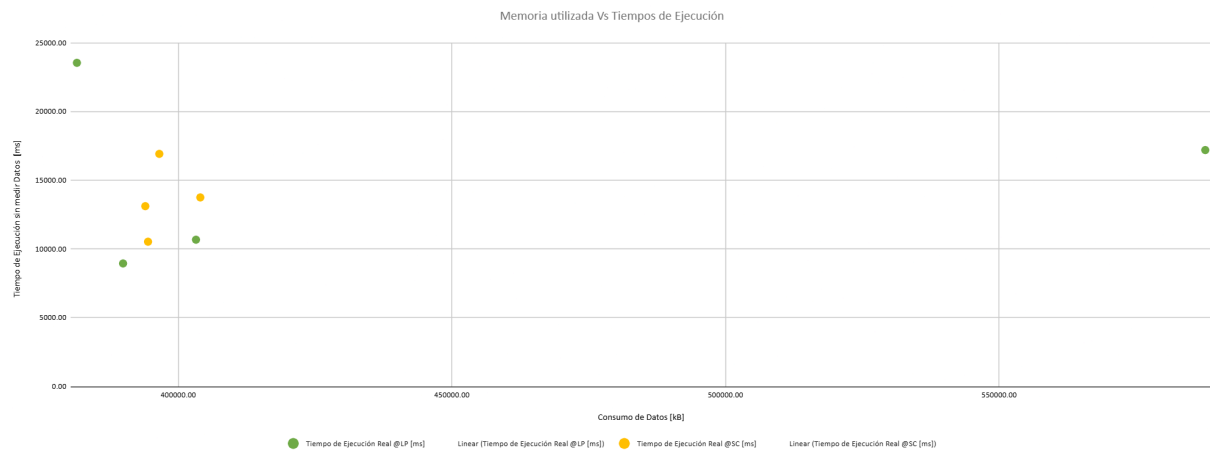
| Factor de Carga (CHAINING) | Consumo de Datos [kB] | Tiempo de Ejecución Real @SC [ms] |
|----------------------------|-----------------------|-----------------------------------|
| 2.00                       | 404020.67             | 13744.50                          |
| 4.00                       | 396526.93             | 16911.12                          |
| 6.00                       | 394466.82             | 10525.39                          |
| 8.00                       | 393968.36             | 13109.41                          |

Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING

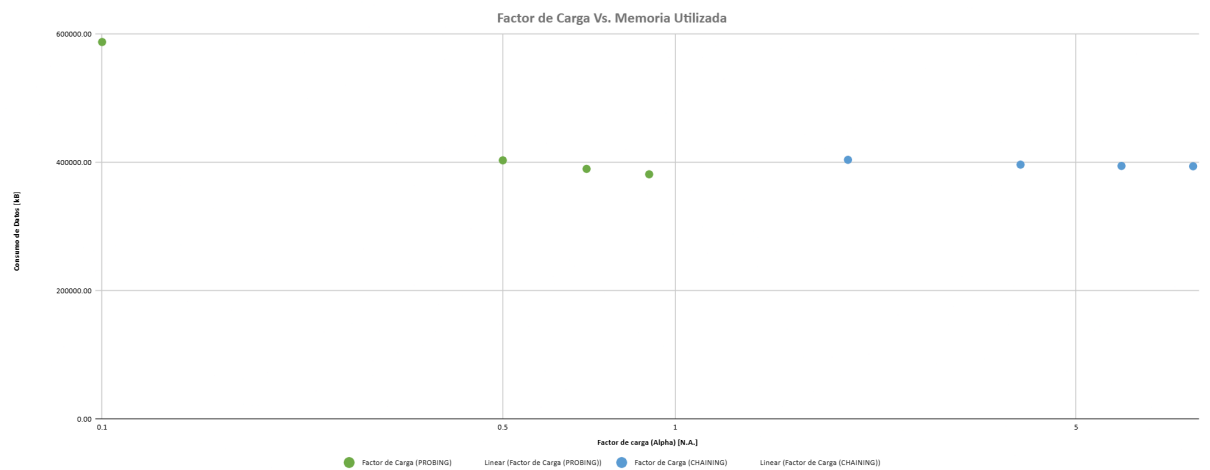
## Gráficas

La gráfica generada por los resultados de las pruebas de rendimiento.

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING



- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING



## Preguntas de análisis

1. ¿Por qué en la función **getTime()** se utiliza **time.perf\_counter()** en vez de otras funciones como **time.process\_time()**?

`perf_counter()` mide el tiempo real que pasa, como un cronómetro, incluyendo cuando el programa está esperando, por ejemplo mientras lee los CSV del disco. `process_time()` solo cuenta el tiempo en que la CPU estuvo trabajando en el proceso, así que daría un tiempo menor al que de verdad tarda la carga. Como lo que nos interesa es cuánto tarda realmente, tiene más sentido usar `perf_counter()`.

2. ¿Por qué son importantes las funciones **start()** y **stop()** de la librería **tracemalloc**?

`tracemalloc` solo registra la memoria mientras está activo. Con `start()` lo prendemos justo antes de la operación, para medir solo lo que ella usa, y con `stop()` lo apagamos y se libera la memoria que usa para guardar sus registros. Si no se apaga, el programa sigue más lento y las siguientes mediciones se mezclan con las anteriores.

3. ¿Por qué no se puede medir paralelamente el **uso de memoria** y el **tiempo de ejecución** de las operaciones?

Porque `tracemalloc` hace más lento el programa: cada vez que se reserva memoria tiene que anotarlo. Si medimos el tiempo con `tracemalloc` prendido, ese trabajo extra se suma y ya no es el tiempo real. Por eso con `memflag` medimos el tiempo en una ejecución y la memoria en otra.

4. Dado el número de elementos de los archivos (`large`), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?

El factor de carga es el número de elementos dividido por el tamaño de la tabla. Se cargaron 10.000 libros, 5.841 autores y 34.252 tags, y como la tabla hace rehash cuando se pasa del límite, el factor real queda casi igual al configurado. Por ejemplo, con LP 0.7 los libros quedan en  $10.000 / 14.293 = 0.70$  y con SC 4 en  $10.000 / 2.503 = 4.00$ . En linear probing tiene que ser menor a 1 porque cada casilla guarda un solo elemento, mientras que en separate chaining puede ser mayor porque cada casilla es una lista.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?

El caso que más se notó fue LP con 0.9, que fue el más lento: 23.530 ms en promedio, y hasta su mejor corrida tardó más del doble que las mejores de los demás (unos 8.500 ms). Esto pasa porque con la tabla casi llena se forman clusters y `find_slot` tiene que revisar muchas casillas antes de encontrar una libre. En los demás casos los tiempos cambiaron mucho de una corrida a otra, así que no se alcanza a ver una tendencia clara.

6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?

A mayor factor de carga, menos memoria, porque la tabla tiene menos casillas vacías. En LP se nota mucho: de 587.664 kB con 0.1 a 403.232 kB con 0.5, porque con 0.1 hay unas 10 casillas por cada elemento. De ahí en adelante baja poco. En SC casi no cambia (de 404.021 a 393.968 kB), porque solo se

ahorran listas vacías y la mayoría de la memoria son los datos de los libros, que son los mismos en todos los casos.

7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

Sin contar LP 0.9, no vimos una diferencia clara: los mejores tiempos de los dos esquemas fueron casi iguales (entre 8.300 y 9.000 ms). La diferencia está en qué pasa cuando la tabla se llena: LP se vuelve muy lento cerca de 1 por los clusters, mientras que en SC solo se alargan un poco las listas, así que aguanta factores más altos sin volverse mucho más lento.

8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

La memoria fue parecida en los dos: LP entre 0.5 y 0.9 usó entre 381.471 y 403.232 kB, y SC entre 393.968 y 404.021 kB. SC usa muchas menos casillas, pero cada elemento necesita un nodo extra y cada casilla es una lista. LP no tiene nodos, pero sí necesita casillas vacías, así que se compensa. La única diferencia grande fue LP 0.1, por la cantidad de casillas vacías. LP 0.9 fue la que menos memoria usó, pero también la más lenta.